

MAESTRO AGRÍCOLA

VERSÃO TÉCNICA REVISADA

Equipe AgroTurtles

AI Glasses Brasil 2026

Revisão: 22 de agosto de 2026



Versão técnica revisada

Equipe: AgroTurtles Trilha temática: Produtividade Área de aplicação: Agronegócio Revisão: 22 de agosto de 2026

Equipe

- Átila Capozzoli Ribeiro Rodrigues - Desenvolvedor Fullstack Pleno com experiência em apps Kotlin e React Native; lidera o app Android nativo, o DAT, áudio e a máquina de estados.
- Felipe Gonçalves Vidal - Estudante de Ciência da Computação (INF/UFG) e integrante do Núcleo de Robótica Pequena Mecânica; lidera visão computacional, ROS 2, Gazebo, TurtleBot 4 e integração com o simulador.
- Rafael José de Souza Marques - Estudante de Ciência da Computação (INF/UFG) e voluntário no CEIA; lidera IA local, classificador de intenção, conjunto de testes e métricas do modelo.

Felipe e Rafael apresentam o pitch. O MVP mantém um único app Android nativo em Kotlin, com contrato e modelo local canônicos. React Native fica fora do escopo.

Revisão de viabilidade: o escopo “olhar, falar e confirmar” foi preservado. Foram removidas dependências de pose/IMU e profundidade que não estão disponíveis na superfície atual do DAT. O MVP usa um alvo visual mapeado e mantém a localização métrica no lado ROS.

1. Resumo executivo

O Maestro Agrícola é uma interface hands-free para comandar robôs agrícolas autônomos. O operador centraliza um alvo visual, expressa a ação por voz e confirma o comando por áudio. Um app companion nativo Android interpreta a intenção e envia um comando versionado a um bridge ROS 2, que associa o alvo a uma pose conhecida e aciona a navegação no Gazebo.

O produto não implementa navegação pesada, desvio de obstáculos ou segurança funcional do veículo. Essas responsabilidades continuam no robô. O Maestro atua como camada de interação, validação e orquestração.

2. Problema e público-alvo

2.1 Problema

A operação de robôs móveis no campo ainda pode exigir telas, menus e coordenadas. Em ambientes com sol, poeira, luvas e deslocamento constante, essa interação interrompe o trabalho e aumenta a barreira técnica.

2.2 Usuários iniciais

- Operadores de robôs móveis e tratores autônomos.
- Equipes de agricultura de precisão.
- Laboratórios e integradores que utilizam ROS 2.
- Fabricantes capazes de receber comandos por API ou gateway.

2.3 Hipótese de valor

Uma interface “olhar, falar e confirmar” pode reduzir a fricção e manter a atenção do operador no ambiente. A hipótese será validada com métricas de tempo, erros, cancelamentos e compreensão do feedback; não são assumidos ganhos percentuais antes dos testes.

3. Objetivo e não objetivos do MVP

3.1 Objetivo

Demonstrar de ponta a ponta que um operador consegue selecionar um alvo mapeado por visão ou ID falado, declarar uma intenção por voz, confirmar por áudio e produzir movimento em um robô simulado.

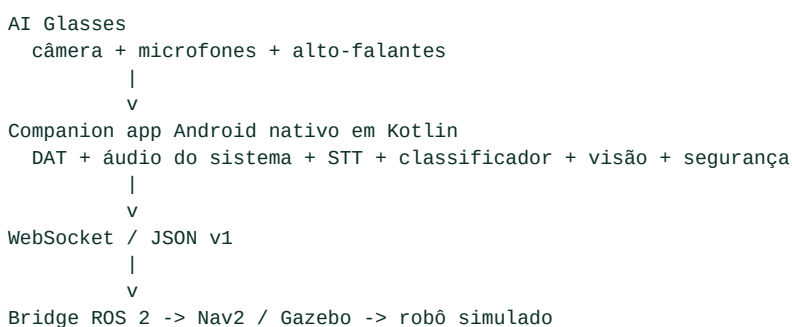
3.2 Não objetivos

- Navegação, path planning e desvio de obstáculos.
- Pulverização real ou controle direto de atuadores perigosos.
- Precisão centimétrica, RTK ou mapeamento da fazenda.
- Operação com múltiplos robôs.
- Linguagem natural aberta para qualquer tarefa.
- Inferência de coordenada baseada em IMU ou pose dos óculos.
- Certificação de segurança para maquinário real.

4. Jornada crítica

- 1 O operador centraliza a placa legível PLOT-03, que também contém o QR do talhão.
- 2 O app mantém ou inicia a sessão DAT e obtém a foto necessária; no corte pré-hardware, a fonte é o MockDeviceKit explicitamente habilitado.
- 3 O operador diz “pulverizar esta área” ou explicita “pulverizar no plot-03”.
- 4 O app transcreve a fala e classifica a intenção como SPRAY.
- 5 O resolvidor combina o ID visual e o ID falado: concordância seleciona p l o t - 03; divergência cancela.
- 6 O sistema diz: “pulverizar talhão três, confirmar?”.
- 7 O operador responde “confirmar”.
- 8 O app envia o comando por WebSocket.
- 9 O bridge valida schema, expiração e duplicidade.
- 10 O bridge mapeia p l o t - 03 para uma pose conhecida e publica a meta no ROS 2.
- 11 Se necessário, um UNDOCK explícito e confirmado libera a saída da doca; SPRAY nunca faz undock implícito.
- 12 O robô simulado inicia o deslocamento e permanece no alvo após a conclusão do Nav2.
- 13 O sistema informa “comando enviado”; o retorno à doca só ocorre após DOCK explícito e confirmado.

5. Arquitetura



5.1 Óculos

- Câmera acessada pelo DAT mediante registro, permissão, sessão e stream.
- Microfones e alto-falantes dependem do caminho de áudio nativo do Android; a rota Bluetooth dos óculos ainda não foi validada.
- Sem display; todos os estados relevantes precisam de feedback sonoro.
- Sem processamento de negócio nos óculos.

5.2 Companion app

Responsabilidades:

- inicializar o SDK e observar registro, dispositivo, sessão e stream;
- rotear e capturar voz;
- obter frame ou foto do alvo;
- rodar STT, classificador de intenção e detector de alvo;
- extrair um ID de plot falado e compará-lo com o alvo visual;
- controlar confirmação, recusa, timeout e erros;
- emitir TTS e sinais sonoros;
- enviar comandos idempotentes;
- registrar telemetria sem mídia bruta.

5.3 Bridge ROS 2

- expor endpoint WebSocket local;
- validar `schema_version`, campos e tipos;
- rejeitar comandos expirados, não confirmados ou duplicados;
- mapear `target.id` para uma pose do cenário;
- publicar a meta de navegação;
- devolver `ACCEPTED`, `REJECTED` ou `FAILED`.

6. Decisão de localização do alvo

6.1 Premissa removida

A versão anterior propunha cruzar pixel central, pitch/yaw dos óculos e GPS do smartphone para realizar raycasting. Essa solução depende de pose da câmera sincronizada e calibrada. A superfície pública atual do DAT documenta câmera, foto, sessão e mock, mas não fornece a pose/IMU necessária para essa conta.

Além disso, a orientação do smartphone não substitui a orientação dos óculos quando o telefone está no bolso ou em outra posição. Esse fallback foi removido.

6.2 Implementação do MVP

O cenário contém uma placa vertical com o texto `PL0T-03` e um QR previamente mapeado. O frame retorna `target_id`; o bridge ROS conhece a pose correspondente. O texto humano permite que o operador diga o mesmo ID caso o marcador esteja parcialmente obstruído.

Regras:

- considerar apenas alvo dentro de uma região central da imagem;
- exigir exatamente um candidato com confiança acima do limiar;

- aceitar plot-03, plot três, plot zero três ou talhão três como o mesmo ID falado;
- exigir concordância quando voz e câmera identificarem alvos simultaneamente;
- pedir reposicionamento quando houver zero ou múltiplos candidatos;
- nunca transformar alvo desconhecido em coordenada padrão.

O caminho principal continua visual: “pulverize aqui” exige QR. O ID falado é um fallback e ainda passa por repetição em áudio e confirmação explícita. Um conflito entra em AMBIGUOUS, limpa o alvo resolvido e não cria comando.

6.3 Poeira, localização do operador e robustez

O QR do MVP prova identificação, não robustez industrial. Uma implantação rural exigiria placa maior, selada e lavável, correção de erro, cópias redundantes, posicionamento que reduza acúmulo de poeira e rotina de inspeção.

GPS do celular informa aproximadamente onde o operador está, não para onde olha nem qual destino deseja. Ele pode futuramente limitar candidatos quando os talhões tiverem polígonos georreferenciados, mas não substitui pose da câmera ou telemetria do robô. O Maestro nunca converte a posição do usuário em destino padrão.

6.4 Evolução de produto

Possíveis substitutos futuros para o QR:

- localização visual contra mapa da fazenda;
- marcos semânticos reconhecidos por visão;
- mapa e pose fornecidos pelo robô;
- RTK, UWB ou infraestrutura local;
- futura API oficial de pose, se disponibilizada.

Essas opções exigem validação separada e não entram na demonstração principal.

7. Voz e inteligência artificial

7.1 Captura e transcrição

No Android atual, o app solicita reconhecimento de fala com preferência offline e reproduz TTS pelo caminho de saída ativo. O código não seleciona explicitamente um dispositivo Bluetooth; essa configuração e a rota dos óculos permanecem como gate de hardware.

O STT fica atrás de uma interface e o MVP exige execução on-device quando o pacote de idioma estiver disponível. Caso um aparelho não ofereça reconhecimento offline, o plano de contingência para desenvolvimento é entrada textual simulada, sem enviar áudio a um serviço externo. O classificador de intenção continua sempre local.

7.2 Classificador de intenção

O componente de IA comprovável do MVP é uma cascata híbrida local e compacta: regras de alta precisão tratam negação, hesitação e comandos inequívocos; um classificador linear softmax multiclasse resolve as demais frases; baixa confiança resulta em UNKNOWN. O treino canônico registra 213 exemplos e seis rótulos. O artefato JSON tem 729.056 bytes. Entrada: transcrição curta. Saída:

```
{
  "intent": "SPRAY",
  "confidence": 0.93,
  "source": "MODEL"
}
```

Rótulos atuais:

- SPRAY
- DOCK

- UNDOCK
- CONFIRM
- CANCEL
- UNKNOWN

O texto é normalizado para caixa e acentuação; as features incluem palavras, bigramas e n-gramas de caracteres de três a cinco posições. A mesma política e os mesmos pesos são interpretados em Kotlin, e a saída informa origem RULE ou MODEL. Com limiar operacional de 0,40, os 64 casos da avaliação original foram classificados corretamente, com macro-F1 1,00 e zero aceite perigoso; esse corpus cobre as quatro classes originais. O corpus de campo cobre os seis rótulos e obteve 48/48 na regressão reproduzível. Ambos descrevem suítes controladas, não desempenho no campo.

Uma evidência física anterior registra 540 inferências no Motorola Edge 40 Neo, mediana de 229 μ s, p95 de 1.013 μ s e zero divergências, mas os hashes mostram que ela pertence a um modelo e APK anteriores. O candidato atual exige nova coleta antes de congelar métricas de dispositivo.

O Qwen2.5-1.5B Q4_K_M é uma camada conversacional opcional executada via `llama.cpp`. O benchmark versionado como classificador operacional obteve 36/48, macro-F1 0,7384 e três aceites perigosos. Por isso, o `LanguageRouter` envia ao Qwen somente entradas já classificadas como UNKNOWN, e sua saída é limitada a CHAT ou OUT_OF_SCOPE; ele não conhece Command, TargetResolver, WebSocket ou ROS.

O STT não é esse modelo: o Android tenta a transcrição on-device por sua API nativa. O classificador do Maestro recebe somente o texto e não envia áudio ou transcrição para um servidor de inferência.

7.3 Regras de segurança sobre a IA

- confiança abaixo do limiar resulta em UNKNOWN;
- intenção desconhecida nunca produz movimento;
- regras determinísticas priorizam cancelamento e bloqueiam ambiguidades antes de aceitar ordens positivas;
- o usuário sempre ouve a interpretação antes de confirmar.

8. Câmera e áudio simultâneos

Câmera e áudio não devem ser tratados como uma única API. O DAT fornece o caminho de câmera; o áudio segue as APIs nativas de Android e o comportamento pode variar por smartphone, sistema, versão do SDK, firmware e ambiente de rádio.

Plano de validação:

- 1 Manter o DAT 0.9.0 fixado até uma atualização deliberada.
- 2 Configurar e registrar o dispositivo de comunicação antes de iniciar a captura.
- 3 Evitar renegociar o canal de áudio durante a sessão.
- 4 Começar com qualidade LOW ou MEDIUM e 7 fps.
- 5 Testar voz + câmera por cinco minutos no aparelho real.
- 6 Medir frames recebidos, quedas, latência e erros de sessão.
- 7 Ajustar qualidade somente com evidência.

O desenvolvimento sem hardware usa Mock Device Kit para câmera e um fone Bluetooth comum para áudio. Isso valida a lógica, mas não substitui o teste final nos óculos.

9. Contrato de comunicação

9.1 Comando

```
{
  "schema_version": "1.0",
  "command_id": "9f9a8ef8-94b9-4f4e-a436-92357e5a7a20",
  "created_at": "2026-08-16T15:00:00Z",
  "expires_in_ms": 5000,
  "intent": "SPRAY",
  "target": {
    "type": "MAPPED_PLOT",
    "id": "plot-03"
  },
  "confirmed": true
}
```

9.2 Resposta

```
{
  "command_id": "9f9a8ef8-94b9-4f4e-a436-92357e5a7a20",
  "status": "ACCEPTED"
}
```

9.3 Regras

- `command_id` é UUID e chave de deduplicação.
- `confirmed` precisa ser `true`.
- Comando vencido é rejeitado.
- `target.id` precisa existir no mapa ativo.
- Reenvio retorna o resultado anterior e não repete movimento.

10. Máquina de estados

IDLE

```
-> CAPTURING ou ID_FALADO
-> INTERPRETING
-> AWAITING_CONFIRMATION
-> SENDING
-> ACCEPTED
-> IDLE
```

Qualquer etapa pode terminar em:

AMBIGUOUS | TARGET_CONFLICT | CANCELLED | TIMEOUT | CONNECTION_ERROR

Somente `AWAITING_CONFIRMATION` -> `SENDING` aceita uma confirmação válida. Frases ou eventos recebidos fora desse estado são ignorados ou tratados como nova interação.

11. Checkpoints do programa

Checkpoint	Evidência na demonstração
Inteligência artificial	Classificador local retorna intenção e confiança
Câmera ou microfone	DAT 0.9.0 + MockDeviceKit identifica o alvo no pré-hardware; câmera e áudio físicos são o próximo gate
Output por áudio	TTS confirma, informa erros e encerra a jornada
Privacidade e dados	Nenhuma mídia persistida pelo app; fluxo de terceiros documentado
Eficiência de bateria	Captura sob demanda ou baixa taxa; recursos encerrados ao final

12. Privacidade, dados e ética

12.1 Dados do Maestro

- Frames e áudio existem somente durante a interação.
- O app não grava galeria, arquivo de áudio ou transcrição por padrão.
- Após inferência, referências à mídia são liberadas.
- Logs contêm estado, latência, `target_id`, intenção e erro, sem mídia bruta.

“Não persistir” não significa apagar fisicamente cada cópia de RAM mantida pelo sistema operacional ou SDK. A promessa verificável é que o código do Maestro não cria armazenamento persistente de mídia.

12.2 Plataforma e SDK

Android, Meta AI e DAT podem processar informações necessárias para pareamento, permissões, funcionamento e telemetria. A equipe deve:

- documentar esses fluxos na política de privacidade;
- ativar o opt-out de analytics opcionais do DAT quando permitido;
- solicitar somente permissões necessárias;
- informar claramente quando algum provedor externo for usado.

12.3 Segurança operacional

- confirmação humana obrigatória;
 - expiração e deduplicação de comandos;
 - ausência de execução em ambiguidade;
 - robô mantém geofencing, parada de emergência e desvio de obstáculos;
 - o MVP não é certificado para controlar maquinário real.
-

13. Eficiência de bateria e desempenho

- Evitar streaming contínuo quando uma captura atende à jornada.
 - Se o stream for necessário, começar em LOW/MEDIUM e baixa taxa.
 - Limitar inferência a uma janela curta após o comando.
 - Encerrar áudio, stream e sessão quando não forem mais necessários.
 - Medir latência, temperatura e bateria no hardware real.
 - Não anunciar metas numéricas sem benchmark reproduzível.
-

14. Testes e critérios de aceite

14.1 Casos funcionais

- comando válido e confirmado;
 - recusa explícita;
 - intenção ambígua;
 - alvo ausente;
 - múltiplos alvos;
 - ID falado sem QR;
-

- concordância e divergência entre ID falado e QR;
- alvo falado fora do mapa;
- timeout de confirmação;
- conexão perdida;
- comando duplicado;
- comando expirado.

14.2 Critérios EARS

- 1 Quando o operador iniciar um comando, o sistema deve obter somente a mídia necessária para resolver o alvo.
- 2 Quando intenção e alvo forem válidos, o sistema deve pedir confirmação por áudio antes do envio.
- 3 Se alvo ou intenção forem ambíguos, então o sistema deve pedir repetição e não enviar movimento.
- 4 Se a confirmação não chegar no prazo, então o sistema deve cancelar a operação.
- 5 Quando o bridge aceitar o comando, o sistema deve informar sucesso por áudio e registrar somente telemetria sem mídia bruta.

14.3 Definição de pronto

A jornada crítica precisa rodar cinco vezes seguidas no cenário limpo, incluindo pelo menos uma recusa e uma ambiguidade. Não pode haver movimento indevido, duplicado ou iniciado sem confirmação.

15. Riscos e planos de contingência

Risco	Mitigação	Plano B
Mudança no DAT	Fixar versão validada e encapsular SDK	Voltar ao último build aprovado
Sem pose/IMU	Alvo visual mapeado	Seleção manual no app apenas para diagnóstico
Poeira ou QR obstruído	Placa legível, grande e redundante	ID falado validado e confirmação reforçada
Voz + câmera instáveis	Roteamento antes do stream e qualidade reduzida	Captura sequencial, mantendo câmera como checkpoint
Modelo lento	Reduzir atributos ou vocabulário	Usar regras apenas como validação de segurança
Hardware apenas no evento	Mock Device Kit e feeds gravados	Demo reproduzível com mock
ROS indisponível	Bridge com simulador de resposta	Vídeo da jornada pré-validada, identificado como contingência

16. Plano de desenvolvimento

Antes do hackathon

- [x] compilar e instalar `datDebug` em Android físico com o `MockDeviceKit` explicitamente habilitado;
- [x] integrar QR local por ZXing às fotos entregues pela stack DAT;
- [x] fechar a jornada Android WebSocket ROS 2/Nav2/Gazebo com lifecycle explícito;
- [x] manter testes automatizados do contrato, da IA, do bridge e dos estados;
- [] gravar e ensaiar o pitch final de sete slides.

No hardware real

- parear, conceder permissões e validar câmera e áudio separados;
 - validar câmera e áudio simultâneos;
 - medir latência e estabilidade e congelar a configuração aprovada;
 - rodar a jornada cinco vezes e gravar evidências.
-

17. Diferencial e caminho de produto

O Maestro Agrícola não é apenas um assistente que descreve o campo. Ele transforma visão e voz em um pedido operacional confirmado, mantendo a autonomia e as proteções no robô.

A oportunidade de 20% a 30% exibida no pitch é uma hipótese de redução de custo operacional e retrabalho a validar em piloto controlado. Ela não é tratada como resultado técnico deste MVP.

Após o MVP, o produto pode evoluir de alvos mapeados para localização visual mais natural, ampliar intenções e integrar plataformas diferentes pelo mesmo contrato. Cada evolução será validada sem alterar a regra central: o humano decide a ação e confirma; a máquina executa dentro de seus próprios limites de segurança.

18. Referências principais

- [DAT Android](#) e [CameraAccess Android](#).
- [Getting started toolkit](#).
- Edital AI Glasses Brasil 2026.
- Materiais de apoio Meta, Unidades XII, XIII e XIV.